Lecture 9: Peripherals

Lui Holder-Pearson

September 24, 2026

Learning Goals

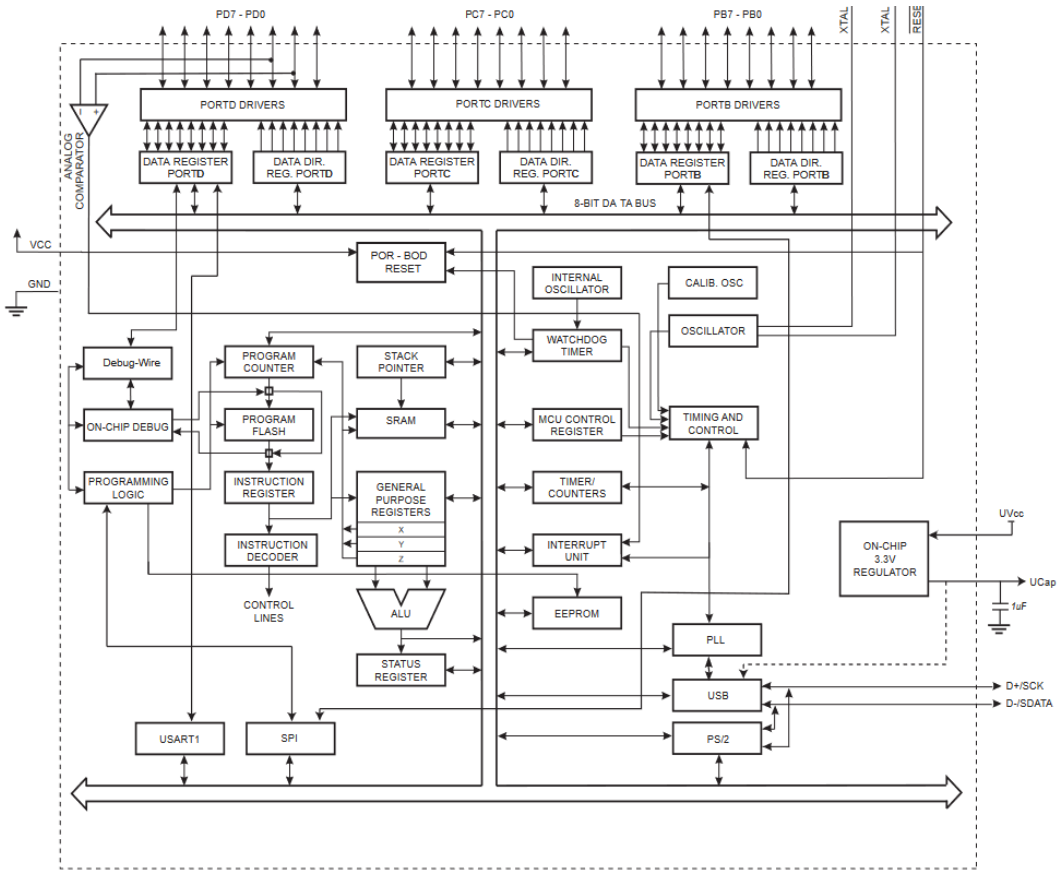
- Understand the need and the use of peripherals within microcontrollers
- Understand the use of timer and counter peripherals with the ATMega32u2.
- Understand how analogue signals are estimated with the ATMega32u2:
  - As inputs using the analogue comparator, and
  - As outputs using pulse-width modulation.

Further Reading

ATMega32u2 datasheet
Youtube video discussing timers and counters in general
Youtube video explaining the specific registers more in detail

1 Internal peripherals

There are dedicated circuits within microcontrollers that provide significant expanded functionality beyond GPIO.



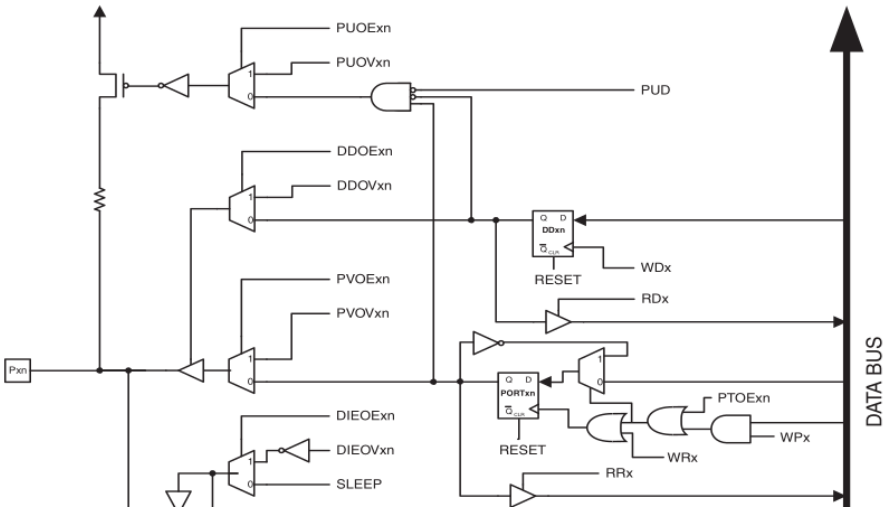
Specific peripherals that are able to be connected to a GPIO pin are referred to as the *alternate port function*. The selection is performed through multiplexing the output:

Table 12-3. Port B Pins Alternate Functions

Port Pin	Alternate Functions
PB7	OC0A/OC1C/PCINT7 (Output Compare and PWM Output A for Timer/Counter0, Output Compare and PWM Output C for Timer/Counter1 or Pin Change Interrupt 7)
PB6	PCINT6 (Pin Change Interrupt 6)
PB5	PCINT5 (Pin Change Interrupt 5)
PB4	T1/PCINT4 (Timer/Counter1 Clock Input or Pin Change Interrupt 4)
PB3	PDO/MISO/PCINT3 (Programming Data Output or SPI Bus Master Input/Slave Output or Pin Change Interrupt 3)
PB2	PDI/MOSI/PCINT2 (Programming Data Input or SPI Bus Master Output/Slave Input or Pin Change Interrupt 2)
PB1	SCLK/PCINT1 (SPI Bus Serial Clock or Pin Change Interrupt 1)
PB0	$\overline{\text{SS}}$/PCINT0 (SPI Slave Select input or Pin Change Interrupt 0)

Peripherals are also used for specific communication means, allowing specific hardware to be used to interpret signals, rather than forcing high-speed, high-fidelity, synchronous software to be required.

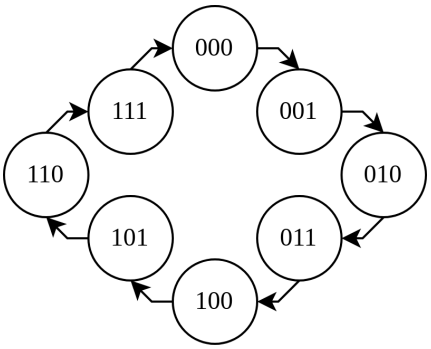
Figure 12-5. Alternate Port Functions(1)



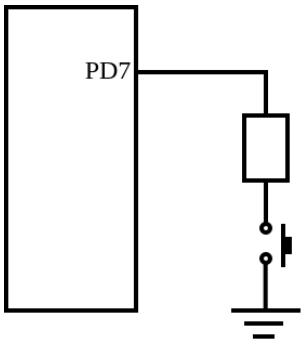
The complete schematic block diagram can be found in the datasheet in Figure 13-5, page 63.

2 Timers and counters

Recall from Lecture 8: Sequential Logic Circuits the counter circuit. The counter circuit is a very simple finite state machine, which increments the binary value by one every state change. Specifically, the state change is triggered by a rising edge on the input clock signal.



What is the functional behaviour of the counter circuit when the `CLK` signal is connected to:



- A clock signal?
- A button push?
- An event logger?

2.1 Timers

There are several counters within any microcontroller. They provide the ability for the microcontroller to estimate the real passing of time. This is vital for any microcontroller, to varying degrees of accuracy. Some specific uses of timers include:

- Watchdog timers
- Communication protocols
- Real world inputs and outputs
- Timekeeping
  - For the programme itself
  - As a watch, for example

For counters to act as counters, a clock signal from a specific oscillatory source is provided to the counter.

Timers do not need an external signal, but instead can have an internal signal. This allows generation of interrupts.

Case study: LED flashing: Option I

```
1  void delay_ms (int wait_time) {
2    int time = 0;
3    ...
4    while (time < (wait_time * ms_scale_factor)) {
5      time++;
6    }
7
8  int main (void) {
9    ...
10   while (1) {
11     GPIO_toggle(LED_pin);
12     delay_ms(500);
13   }
14 }
```

Bonus Exercise: What is the most efficient way to toggle the LED pin? (Refer to the shaded portions of the complete GPIO schematic diagram from Lecture 9).

Toggling the LED should only take a handful of instructions, therefore the remainder of time should be able to be spent performing other tasks, such as reading inputs, communicating, and processing.

With the above implementation, can we do that?

Case study: LED flashing: Option II

```
ISR (TIMER0_OVF_vect) {
  GPIO_toggle(LED_pin);
}
int main (void) {
  timer_init();
  while (1) {
    ...
  }
}
```

Timer will not be incrementing constantly, when it reaches a value specified within `timer_init`, the interrupt service routine (ISR) will be called. Thus, we have minimised the CPU impact, and allowed for any other desired software to be run.

2.2 ATMega32u2

The ATMega32u2 has two specific timers:

- One 8-bit timer/counter with *separate prescaler* and *compare* mode (two 8-bit *PWM channels*).
- One 16-bit timer/counter with *separate prescaler*, *compare*, and *capture* mode (three 8-bit *PWM channels*).

What does this actually mean?

- Prescaler
- Compare
- Capture
- PWM Channels

2.3 Interacting with a timer

There are some key registers that we use to interact with the timer/counter unit which let us configure it for use. From section 14 of the ATmega328p datasheet, the key registers for timer 0 are:

- TCNT0

- TCCR0A & TCCR0B

TCCR0B – Timer/Counter Control Register B

Bit	7	6	5	4	3	2	1	0
0x25 (0x45)	FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00
Read/Write	W	W	R	R	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

TCCR0A – Timer/Counter Control Register A

Bit	7	6	5	4	3	2	1	0
0x24 (0x44)	COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

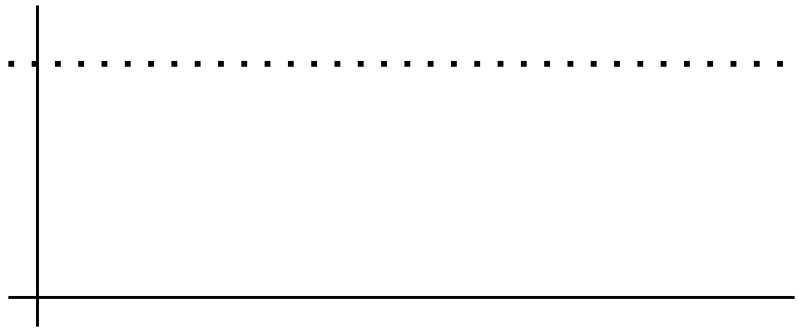
- OCR0A & OCR0B

3 Pseudo analogue signals

The real world contains continuous signals. For an embedded system to interact with the real world, it is necessary to be able to estimate these analogue signals, and be able to output signals that act in an analogue manner.

3.1 Pulse width modulation

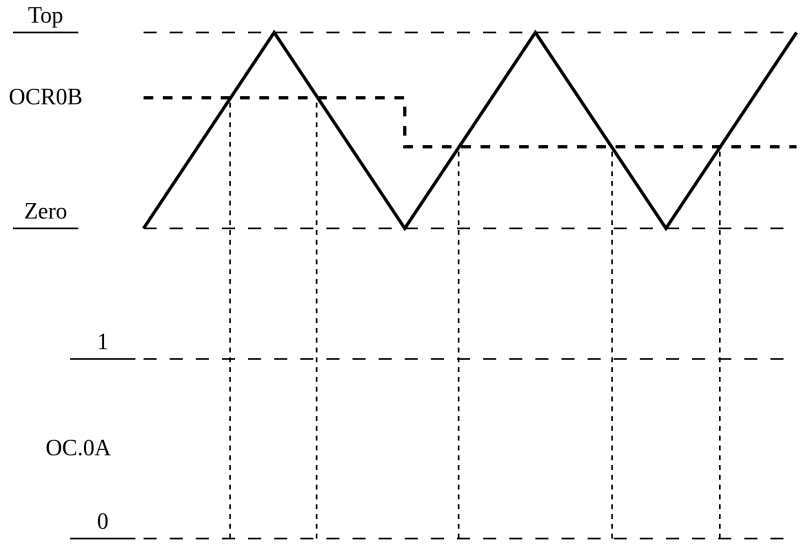
True to its name, PWM is the modulation of the width of a pulse. There are three aspects that define a PWM signal:



- Voltage (amplitude)
- Period
- Duty cycle

- PWM is incredibly useful because it emulates an analogue signal, performing a *digital to analogue conversion* (DAC).
- The average PWM signal is simply proportional to the amplitude multiplied by the duty cycle.
- For example, PWM is used for dynamically adjusting the brightness of an LED, or smoothly adjusting the speed of a DC motor.

3.1.1 Generation from a timer



The registers involved are:

- `TCNTn`
- `OCRnA`
- `OCRnx`
- `OC.nx`

It is possible to determine the key attributes from studying the registers, and the input clock frequency:

- Voltage (amplitude)

- Period

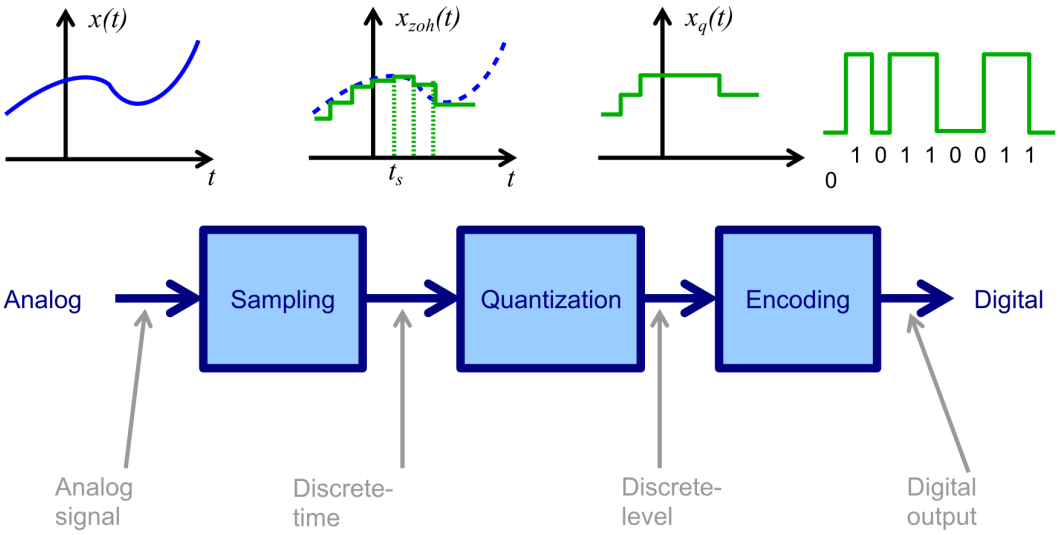
- Duty cycle

For example, what is the frequency and duty cycle if `OCRnA = 0xC0`, $f_{clk} = 10\ kHz$, `OCRnx = 0x30`?

Bonus exercise: What are the fastest and slowest frequencies of PWM output able to be produced by the ATMega32u2, with at least 6-bits of accuracy?

3.2 Analogue input data

Real world signals are continuous in time and amplitude. To estimate the signals, they must be discretised, in terms of both time and the amplitude of the signal. The device/peripheral/subsystem that performs these conversions are called analogue to digital converters (ADCs).



Specifically,

- Sampling:

- Quantisation:

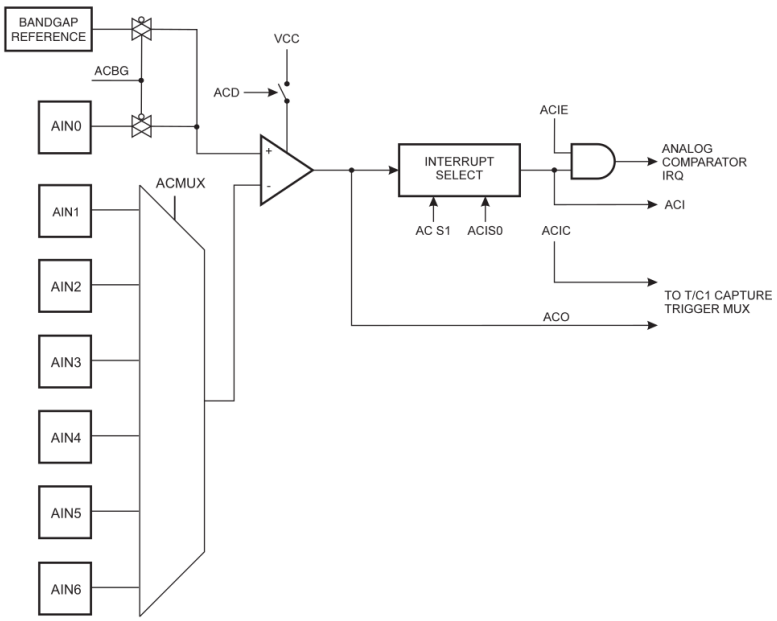
- Encoding:

3.3 ADC with the ATMega32u2

Those of you who are familiar with ATMega328p devices, such as Arduinos, know that there are specific pins that analogue voltages can be read. However, there is no dedicated, specific ADC peripheral within the ATMega32u2.

Instead, an on-board analogue comparator is used:

Figure 22-1. Analog Comparator Block Diagram[(1)]

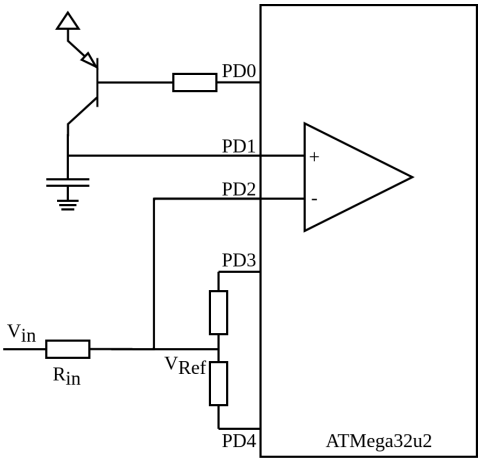


An analogue comparator has an output `=1` if the '+' input is greater than the '-' input, otherwise the output `=0`.

The output of the analog comparator is connected to the TCNT1 capture trigger. This enables storing of the moment in time when the output of the comparator changes state. For example:

For estimation of an analogue voltage, external circuitry is required:

1. Force `PD2 =` V_{ref}:
   - `PD3 = HIGH`
   - `PD4 = LOW`
   - $V_{Cap} = 0\ V$
2. Measure time taken to charge capacitor to $V_{Cap} = V_{Ref}$:
   - Start timer TC1
   - `PD0 = HIGH` to charge V_{Cap}
   - Capture value of TC1 when $V_{Cap} = V_{Ref}$, $t = t_{Ref}$
3. Allow `PD2 =` V_{in}, measure time taken to charge $V_{Cap} = V_{Ref}$:
   - Discharge capacitor.
   - Set `PD3, PD4 =` Hi-Z
   - Start timer TC1 and set `PD0 = HIGH`
   - Capture value of TC1 when $V_{Cap} = V_{in}$, $t = t_{in}$



Because the capacitor charges in a consistent manner, V_{in} can simply be calculated with:

$$V_{in} = V_{Ref} \cdot \frac{t_{in}}{t_{Ref}}$$

Exercises:

1. What is the frequency and duty cycle of a PWM generated with `OCRnA = 0xC0`, $f_{clk} = 10\ kHz$, `OCRnx = 0x30`

Bonus:

1. What is the most efficient way to toggle the LED pin? (Refer to the shaded portions of the complete GPIO schematic diagram from Lecture 9).
2. What are the fastest and slowest frequencies of PWM output able to be produced by the ATMega32u2, with at least 6-bits of accuracy?

Summary

- Understand the need and the use of peripherals within microcontrollers.
  → Internal peripherals allow specific functionality to be implemented in hardware, rather than with expensive, extensive software.
- Understand the use of timer and counter peripherals with the ATMega32u2.
  → Timers and counters within the ATMega32u2 are available. There are two timers, one 8-bit, and one 16-bit.
  → Each have compare and PWM capability, and the 16-bit has capture capability.
- Understand how analogue signals are estimated with the ATMega32u2:
  → Output: through PWM, where the total portion of time a digital square wave is one is a function of register configuration, and thus electronically configurable.
  → Input: through use of an on-board analogue comparator, the time taken to charge a capacitor is measured, and compared to the time take to charge to a known voltage.
  → Subsequently, $V_{in} = V_{Ref} \cdot \frac{t_{in}}{t_{Ref}}$.

Next Lecture: Lecture 10 - Programme execution

4 Questions?